Software Re-Engineering

Lecture 13

Reverse engineering I – Basics and Introduction to assembly language

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Reverse engineering, also called backwards engineering or back engineering, is the process by which an artificial object is deconstructed to reveal its designs, architecture, code, or to extract knowledge from the object.

It’s similar to disassembling the parts of a vehicle to understand the basic functioning of the machine and internal parts etc. and thereafter making appropriate adjustments to give rise to a better performing or superior vehicle.

- In software design, reverse engineering enables the developer or programmer to add new features to the existing software with or without knowing the source code. Different techniques are used to incorporate new features into the existing software.
- Reverse engineering is also very beneficial in software testing, as most of the virus programmers don’t leave behind instructions on how they wrote the code, what they have set out to accomplish etc. Reverse engineering helps the testers to study the virus and other malware code.
- The third category where reverse engineering is widely used is in software security. Reverse engineering techniques are used to make sure that the system does not have any major vulnerabilities and security flaws. The main purpose of reverse engineering is to make the system robust so as to protect it from spywares and hackers. Infact, this can be taken a step forward to Ethical hacking, whereby you try to hack your own system to identify vulnerabilities.

Reverse Engineering Tools

The process of reverse engineering is accomplished by making use of some tools that are categorized into debuggers or disassemblers, hex editors, monitoring and decompile tools

Disassemblers – A disassembler is used to convert binary code into assembly code and also used to extract strings, imported and exported functions, libraries etc. The disassemblers convert the machine language into a user-friendly format. There are different dissemblers that specialize in certain things.

Debuggers – This tool expands the functionality of a disassembler by supporting the CPU registers, the hex duping of the program, view of stack etc. Using debuggers, the programmers can set breakpoints and edit the code at run time. Debuggers analyse the binary in a similar way as the disassemblers and allow the reverser to step through the code by running one line at a time to investigate the results.

With a disassembler you can take a binary and see exactly how it functions (static analysis). Whereas, with a debugger we can step through, break and edit the assembly while it is executing (dynamic analysis).

debugger gives you live execution details, such as variables (like enemy positions on the map), values in memory, etc.

Hex Editors – These editors allow the binary to be viewed in the editor and change it as per the requirements of the software.

Decompile-To decompile means to convert executable or ready-to-run program code -- sometimes called object code -- into some form of higher-level programming language that humans can easily understand. Decompilation is a type of reverse-engineering that performs the opposite operations of a compiler.

A disassembler generates code in a low-level language, while a decompiler generates code in a higher-level programming language.

- The reverse engineering learning process is similar to that of foreign language acquisition for adults.
- The first phase of learning a foreign language begins with an introduction to letters in the alphabet, which are used to construct words with well-defined semantics.
- The next phase involves understanding the grammatical rules governing how words are glued together to produce a proper sentence.
- After being accustomed to these rules, one then learns how to stitch multiple sentences together to articulate complex thoughts.
- Eventually it reaches the point where the learner can read large books written in different styles and still understand the thoughts therein

- In reverse engineering, the language is the architecture and assembly language.
- A word is an assembly instruction. Paragraphs are sequences of assembly instructions.
- A book is a program. However, to fully understand a book, the reader needs to know more than just vocabulary and grammar.
- These additional elements include structure and style of prose, unwritten rules of writing, and others

Books:

- **Practical Reverse Engineering: X86, X64, ARM, Windows Kernel, Reversing Tools, and Obfuscation** by Alexandre Gazet, Bruce Dang, and Elias Bachaalany
- **The IDA Pro Book, 2nd Edition: The Unofficial Guide to the World's Most Popular Disassembler Second Edition** by Chris Eagle

Basics and background knowledge

x86 and x64

- x86 supports the concept of privilege separation through an abstraction called ring level.
- The processor supports four ring levels, numbered from 0 to 3. (Rings 1 and 2 are not commonly used so they are not discussed here.)
- Ring 0 is the highest privilege level and can modify all system settings.
- Ring 3 is the lowest privileged level and can only read/modify a subset of system settings.
- Hence, modern operating systems typically implement user/kernel privilege separation by having user-mode applications run in ring 3, and the kernel in ring 0

- “byte” refers to a one-byte integer (suffix b),
- “word” refers to a two-byte integer (suffix w), “doubleword” refers to a four-byte integer (suffix l), and
- “quadword” refers to an eight-byte value (suffix q).

Processor operations mostly involve processing data. This data can be stored in memory and accessed from thereon. However, reading data from and storing data into memory slows down the processor, as it involves complicated processes of sending the data request across the control bus and into the memory storage unit and getting the data through the same channel.

To speed up the processor operations, the processor includes some internal memory storage locations, called registers.

Memory Segments

- To reference a memory location its logical address has to be specified. The logical address is written as:

Segment number:offset

- For example, A43F:3487h means offset 3487h within segment A43Fh.

Program Segments

Machine language programs usually have 3 different parts. Each of these parts is stored in different memory segments:

Instructions: This is the code part and is stored in the code segment

Data: This is the data part which is manipulated by the code and is stored in the data segment

Stack: The stack is a special memory buffer outside the CPU that is maintained by the CPU as a temporary holding area for addresses and data. It is organized as Last-In-First-Out (LIFO) buffer and is used by the CPU to implement procedure calls. This data structure is stored in the stack segment

- The segment numbers for the code segment, the data segment, and the stack segment are stored in the segment registers CS, DS, and SS, respectively

- A fourth segment register, the ES, or extra segment is provided for programs that need to access a second data segment

- Segment registers cannot be used in arithmetic operations

Addressing modes

When writing assembly programs, the instructions being written require operands. There are several ways to specify the location of the operands. These are called the addressing modes. Depending on where an operand used in instruction is located, the following are the various addressing modes available.

Register addressing mode: When an operand is in a register, it is called register addressing mode. This is the most efficient way of specifying operands as memory access is not required and operands are within the processor. Note the following example, where both the operands in the instruction are process registers.

```
MOV EAX, ECX
```

In the preceding instruction, the contents of the source operand (ECX) will be copied to the destination operand (EAX).

Contd..

Immediate addressing mode: When data is specified as part of the instruction itself, it is called immediate addressing mode. Note how the following instruction contains a constant value as the second operand.

```
mov EAX, 2
```

In the preceding instruction, the value 2 will be stored in the source operand, which is EAX register.

Memory addressing mode: When an operand refer to a memory address, it is memory-addressing mode. As the name suggests, operating in this mode requires access to the memory. Note the following example, where a value referenced by the address of EAX is being moved into the register EBX.

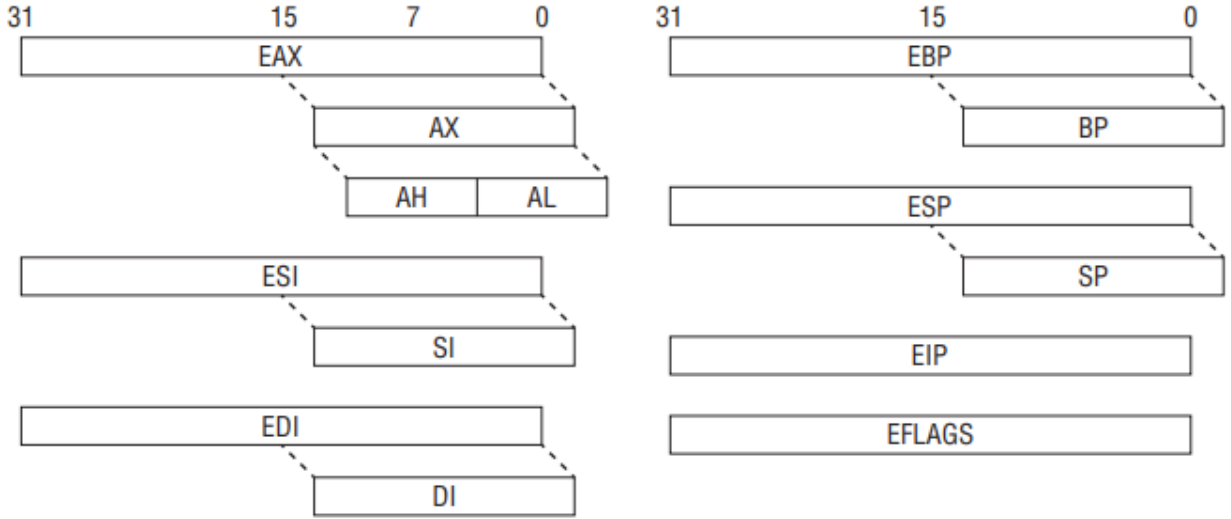
```
MOV EBX, [EAX]
```

Register Set and Data Types

- The x86 architecture provides ten 32-bit registers.
- These registers are grouped into **general purpose, control,** and **segment registers**.
- The general-purpose registers are further divided into **data, pointer,** and **index registers.**
- Data Registers: Data registers are used for arithmetic, logical and other operations. There are four data registers as listed below.

  EAX, EBX, ECX and EDX

It is possible to use a 32-bit register and access the lower half of the data by the corresponding 16-bit register name. For example, the lower 16 bits of EAX can be accessed by using AX. Similarly, the lower two bytes can be individually accessed by using the 8-bit register names. For example, the lower byte of AX can be accessed as AL and the upper byte as AH.



AX (accumulator): This is the accumulator register (it can hold an operand to an arithmetic operation and the result of the operation).

For example: If **1100110001101100** is moved to AX, then the value of AL would be:

0 1 1 0 1 1 0 0

and the value of AH would be:

1 1 0 0 1 1 0 0

MOV AX, 10
ADD AX, AX

In this example, the MOV instruction moves the value 10 into the AX register. The ADD instruction then adds the value in the AX register to itself, effectively doubling it.

BX (base register): This register can hold the address of a procedure or a variable. It can also perform arithmetic and data movement. Again as with the AX, this register can be addressed as a 16-bit value BX, as an 8-bit value BH corresponding to the higher 8-bits of BX, or as BL corresponding to the lower 8-bits of BX.

CX (counter): This register acts as a counter for repeating or looping instructions. These instructions automatically repeat and decrement CX. Similar to AX and BX, it is possible to address the 16-bit value as CX, the upper 8-bits value as CH or the lower 8-bits as CL.

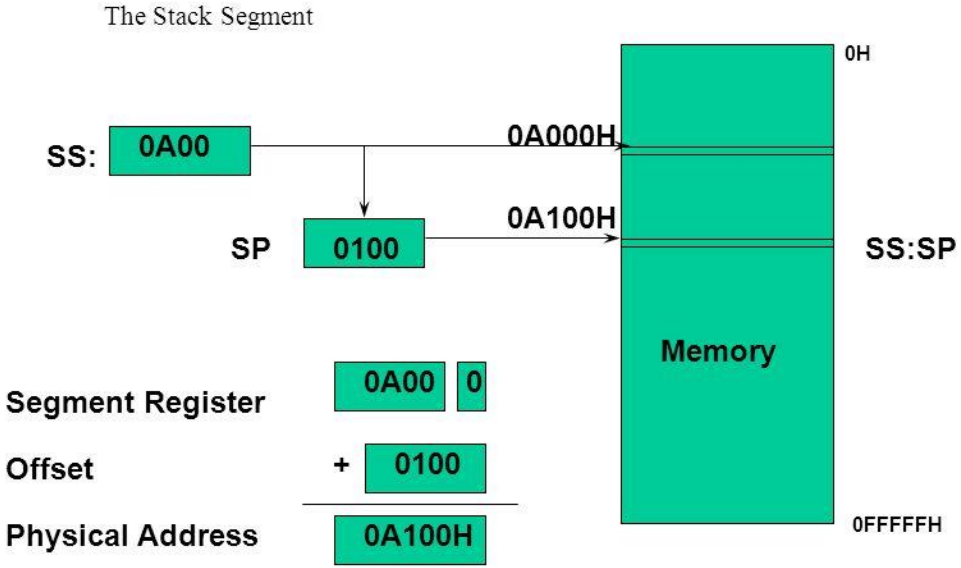
DX (data): This register has a special role in multiply and divide operations. When multiplying, for example, it holds the upper 16-bits of the product. Again, the 16-bit value can be addressed as DX, the upper 8-bits value as DH or the lower 8-bits as DL.

Pointer and Index Registers

- These registers contain the offsets of data and instructions
- The offset refers to the distance of a variable, label, or instruction from its base segment

SP (stack pointer):

- Contains an assumed offset value of the top of the stack
- Combined with the SS (Stack segment) register form the complete logical address of the top of the stack
- The stack is maintained as a LIFO with
  - ❑ its bottom at the start of the stack segment (specified by the SS segment register)
  - ❑ The top of the stack (the location of the last data in the stack) is specified by the offset stored in the SP register



The offset is given by the SP register.
The stack is always referenced with respect to the stack segment register.
The stack grows toward decreasing memory locations.
The SP points to the last or top item on the stack.

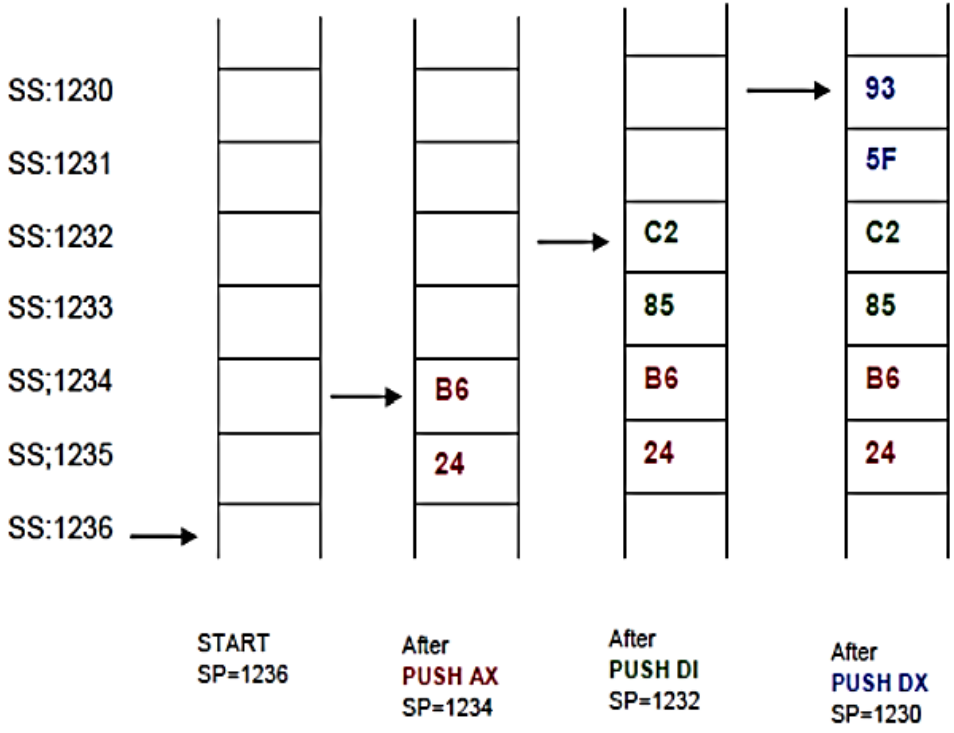
PUSH - pre-decrement the SP
POP - post-increment the SP

BP (base pointer):

- points to the bottom of the stack. This is used to set up and manage information on the stack during a procedure call.
- BP can be used as a base address in the stack area to determine the operand address on the stack

Example: SP=1236H, AX=24B6H, DI=85C2H, and DX=5F93H, show the contents of the stack as each of the following instructions is executed.

```
PUSH    AX
PUSH    DI
PUSH    DX
```



Solution:

Note that in 80x86 the lower byte of the register is stored to the lower address. **(Little Endian Convention)**

SI (source index):

- SI is the source index, used as a pointer to the current character being read in a string instruction (LODS, MOVS, or CMPS). It is also available as an offset to add to BX or BP when doing indirect addressing; for example, the instruction MOV [BX+SI], AX copies the contents of AX into the memory location whose address is the sum of the contents of BX and SI.

DI (destination index):

- DI is the destination index, used as a pointer to the current character being written or compared in a string instruction. It is also available as an offset, just like SI.

IP (Instruction Pointer):

- This is the register used for accessing instructions
- IP register gets updated by the control unit every time an instruction is executed such that it will always point to the next instruction
- Unlike other registers, the IP can not be manipulated by instructions (i.e. it cannot appear as an operand in any instruction)

In terms of **data movement** between registers and memory, the movement can be classified into five general methods:

- Immediate to register
- Register to register
- Immediate to memory
- Register to memory and vice versa
- Memory to memory

- Instructions operate on values that come from registers or main memory. The most common instruction for moving data is MOV. The simplest usage is to move a register or immediate to register.

- Data movement instructions destroy the previous contents of the destination.
- A **load** instruction loads a register from memory.
- A **store** instruction stores the contents of a register into memory.
- A **transfer** instruction loads a register from another register.
- In processors that have separate names for different kinds of data moves, a memory to memory data move might be specially designated as a “move” instruction.

x86 uses square brackets ([]) to indicate memory access

```
02: 8B 08              mov   ecx, [eax]
; set ECX to the value at address EAX
03: 89 18              mov   [eax], ebx
; set the memory at address EAX to EBX
04: 89 46 34           mov   [esi+34h], eax
; set the memory address at (ESI+34) to EAX
05: 8B 46 34           mov   eax, [esi+34h]
; set EAX to the value at address (EAX+34)
06: 8B 14 01           mov   edx, [ecx+eax]
; set EDX to the value at address (ECX+EAX)
```

1. Register to Register

```
mov ax, 10 # move 10 to register 'ax'
mov bx, 5  # move 5 to register 'bx'
add ax, bx
```

2. Memory to Register

```
mov ax, 10  # move 10 to register 'ax'
add ax, [num1] # access value stored at location num1 and store sum
               # again in the register 'ax'
```

3. Register to Memory

```
mov ax, 10
add [num1], ax
```

4. Constant to Register

```
mov ax, 10
add ax, 5 # add 5 to the register 'ax' value and store the sum in 'ax'
```

5. Constant to Memory

```
add [num1], 5 # add 5 to the value stored at num1
```

- The inc and dec instructions can be used to increment or decrement the contents of their respective operands by one
- mul performs an unsigned multiplication of the source operand and the accumulator.\

operation source

- If the source operand is a byte (8 bits), we multiply it by the value stored in the register AL. The result is returned in the registers AH and AL. The higher half of the 16-bit result is stored in AH and the lower half in AL. This means that if the result is small enough to be represented in 8 bits, AH would contain 0.

```
# multiplication - 8 bit source
mov al, 5
mov bl, 10
mul bl      # result in ax

# refreshing registers
mov ax, 0
mov bx, 0
```

operation source

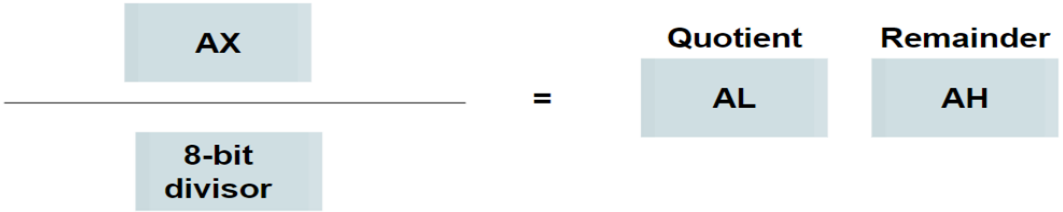
- If the source operand is a word (16 bits), then we multiply it by the value stored in the register AX and the result is returned in the registers DX and AX. The higher half of the 32-bit result is stored in DX and the lower half in AX.

```
# multiplication - 16 bit source
mov ax, 5
mov bx, 10
mul bx      # result in dx:ax

# refreshing registers
mov ax, 0
mov bx, 0
```

- div performs an integer division of the accumulator and the source operand. The result consists of an integer quotient and remainder.

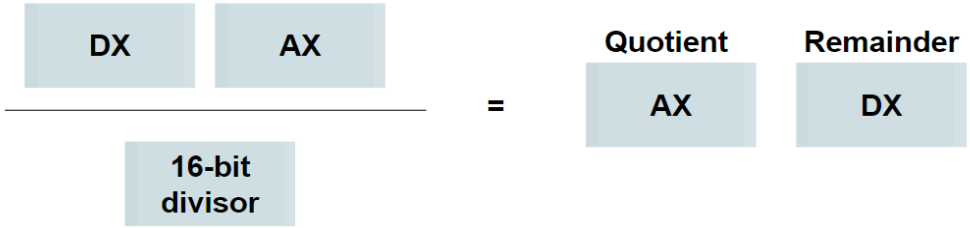
- source operand is a byte, the 16-bit number stored in AX is divided by the operand. The 8-bit quotient is stored in AL and the 8-bit remainder in AH.




```
# divison - 8 bit source
mov al, 23
mov bl, 4
div bl      # quotient in al, remainder in ah

# refreshing registers
mov ax, 0
mov bx, 0
```

- If the source operand is a word, the 32-bit number stored in DX : AX is divided by the operand. The higher half of the number is stored in DX and the lower in AX. The 16-bit quotient is stored in AX and the remainder in DX.




```
# divison - 16 bit source
mov ax, 23
mov bx, 4
div bx      # quotient in ax, remainder in dx

mov ax, 0x4c00  # terminate program
int 0x21
```

a program to show arithmetic instructions in AL

```
# addition
mov ax, 5   # load number in ax
mov bx, 2   # load number in bx
add ax, bx  # accumulate sum in ax

# subtraction
mov cx, 10
mov dx, 3
sub cx, dx  # accumulate difference in cx

# refreshing registers
mov ax, 0
mov bx, 0
mov cx, 0
mov dx, 0
```

Stack Operations and Function Invocation

- Push means to put something on top of the stack;
- pop means to remove an item from the top.
- Concretely speaking, on x86, a stack is a contiguous memory region pointed to by ESP and it grows downwards.
- Push/pop operations are done through the PUSH/POP instructions and they implicitly modify ESP.
- The PUSH instruction decrements ESP and then writes data at the location pointed to by ESP;
- POP reads the data and increments ESP.

push ax ; Pushes the value of the AX register onto the stack

pop ax ; Pops the top value from the stack into the AX register

Conditional Instructions

CMP Instruction

The CMP instruction compares two operands. It is generally used in conditional execution. It is used along with the conditional jump instruction for decision making.

Syntax

CMP destination, source

CMP compares two numeric data fields. The destination operand could be either in register or in memory. The source operand could be a constant (immediate) data, register or memory.

Example

```
CMP DX, 00  ; Compare the DX value with zero
JE  L7      ; If yes, then jump to label L7
.
.
L7: ...
```

```
INC     EDX
CMP     EDX, 10 ; Compares whether the counter has reached 10
JLE     LP1     ; If it is less than or equal to 10, then jump to LP1
```

Unconditional Jump

JMP label

Example

The following code snippet illustrates the JMP instruction −

```
MOV  AX, 00    ; Initializing AX to 0
MOV  BX, 00    ; Initializing BX to 0
MOV  CX, 01    ; Initializing CX to 1
L20:
ADD  AX, 01    ; Increment AX
ADD  BX, AX    ; Add AX to BX
SHL  CX, 1     ; shift left CX, this in turn doubles the CX value
JMP  L20       ; repeats the statements
```

Conditional Jump

Instruction	Description	Flags tested
JE/JZ	Jump Equal or Jump Zero	ZF
JNE/JNZ	Jump not Equal or Jump Not Zero	ZF
JG/JNLE	Jump Greater or Jump Not Less/Equal	OF, SF, ZF
JGE/JNL	Jump Greater/Equal or Jump Not Less	OF, SF
JL/JNGE	Jump Less or Jump Not Greater/Equal	OF, SF
JLE/JNG	Jump Less/Equal or Jump Not Greater	OF, SF, ZF

Instruction	Description	Flags tested
JE/JZ	Jump Equal or Jump Zero	ZF
JNE/JNZ	Jump not Equal or Jump Not Zero	ZF
JG/JNLE	Jump Greater or Jump Not Less/Equal	OF, SF, ZF
JGE/JNL	Jump Greater/Equal or Jump Not Less	OF, SF
JL/JNGE	Jump Less or Jump Not Greater/Equal	OF, SF
JLE/JNG	Jump Less/Equal or Jump Not Greater	OF, SF, ZF

The assembly language uses JMP instruction to implement **loops**. However, the processor set can use the LOOP instruction to implement loops conveniently.

- The loop instruction always extracts the number of iterations from the ECX register.
- The loop instruction decrements the value of ECX and compares it with zero.
- If the value in ECX is not equal to zero, the program jumps to the L1 label; otherwise, the program exits the loop.

```
mov ECX 5    ; store the number of iterations in ECX register
L1:
(loop code)
loop l1
```

Readings:

- **Chapter 1, Practical Reverse Engineering: X86, X64, ARM, Windows Kernel, Reversing Tools, and Obfuscation** by Alexandre Gazet, Bruce Dang, and Elias Bachaalany